
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

VIRTUAL SYSTEMS AND SERVICES

From Hypervisors to the Cloud-Native Stack

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



One idea, six times over: put a layer underneath, and lie to what sits above it

A CPU that is not a CPU, a disk that is not a disk, a network that is not a network — and the cost of each lie.

What this volume covers. The HEC course *Virtual Systems and Services*, modernised. Parts I and II teach the 2017 outline close to its published specification — what a hypervisor is, why x86 resisted virtualization for twenty years, and what Intel and AMD added to hardware to end that. Parts III to VI carry the same ideas forward into containers, Kubernetes, storage and network virtualization, cloud service models, live migration, and confidential computing.

HEC published contents for this course once, in 2017. The 2025 revision drops it and divides its ground between *Cloud Computing* and *Microservices Architecture and Docker Containers*. Appendix F states exactly where the line falls, and why a course that goes one layer below both still earns its place.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

This book has one subject, met six times. A **virtualization layer** sits between something real and something that believes it has the real thing, and keeps the belief intact. Chapters 3 to 7 do it to a processor. Chapters 8 to 11 do it to an operating system. Chapter 12 does it to a disk, Chapter 13 to a network, Chapter 15 to a desktop, Chapter 20 to a graphics card. The machinery differs; the question never does — *what is the lie, who maintains it, and what does maintaining it cost?*

Read it in order. Each layer is built from the one before.

The six parts

Part I — Foundations of Virtualization (Chapters 1–3) fixes the vocabulary and then proves the hard part. Chapter 3 is the theoretical spine of the book: Popek and Goldberg’s conditions, why x86 failed them for two decades, and the two ways round that failure. Everything in Part II is a hardware answer to a problem stated in Chapter 3.

Part II — Hypervisors and Compute Virtualization (Chapters 4–7) is the engine room. What Intel and AMD put in silicon; the four platforms you will actually meet; how a CPU and a gigabyte of memory are divided among guests that each think they own everything; and how a device is shared, emulated or handed over outright.

Part III — Containers and Lightweight Virtualization (Chapters 8–11) changes what is virtualized. Not the hardware, but the operating system’s own view of itself: namespaces, control groups and layered filesystems. Then Docker, then Kubernetes, then the designs that put a hypervisor back underneath a container because isolation mattered more than density.

Part IV — Storage, Network and Cloud Virtualization (Chapters 12–15) applies the same idea to everything that is not a CPU. It ends at the cloud, which is this book’s six layers sold by the hour.

Part V — Management, Performance and Security (Chapters 16–19) is the operations half: what you run to manage a fleet, how a running machine moves between hosts without stopping, which numbers tell you a host is overloaded — and the attacks that exist only because the layer exists.

Part VI — Emerging Hardware and Practice (Chapters 20–23) closes with the parts still moving: accelerator partitioning, encrypted memory and attestation, four case studies, and your project.



Why the theory chapter comes third, and not later. Chapter 3 is the most demanding chapter in the book, and it is deliberately early. Without it, hardware-assisted virtualization is a list of acronyms to memorise; with it, every feature in Chapter 4 has a reason — each one exists because trap-and-emulate could not be made to work on x86. A reader who skips Chapter 3 can still pass the course and will never understand why any of it was built.

The furniture of every chapter

Every chapter has the same parts, in the same order. Learn to read them once.

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics. Use it as a checklist before the examination.
🔗 Before you start	Which earlier chapters this one stands on. If these are shaky, go back; nothing here is improved by pressing on.
🔑 Key terms	The vocabulary introduced. Every one appears in the glossary of Appendix D.
📖 Definition	The canonical statement. Learn these verbatim; this field is full of terms used loosely in marketing and precisely in documentation.
💡 Concept	The judgement call — what an experienced administrator would decide here, and why.
⚠️ Warning	Something that will bite you, usually in production and usually at scale.
🧮 Worked example	Real arithmetic, every step shown. Consolidation ratios, page-walk costs, migration times, availability budgets. Do them with a pen.
❗ Common pitfalls	The mistakes that are actually made, with the symptom each produces.
🔧 Try it yourself	The laboratory. Each is a runnable file in <code>code/</code> , named for its chapter.
❓ Checkpoint	Three questions. Attempt them before reading the answers in Appendix C.
📋 Chapter summary	What to retain.
✍️ Review questions	Examination practice: multiple choice, short answer, applied.

How to study with it

1. Read the outcomes first. They tell you what the chapter is *for*.
2. Read the prose once without stopping, then again with a pen on the worked examples. Every number in this book was computed before it was typed; you should be able to reproduce each one.
3. Run the lab. This is not an optional enrichment: a hypervisor is a thing you develop intuition about by breaking it, and the labs are built to be broken. Overcommit memory until the guest swaps. Fill a thin pool. Migrate a machine while pinging it.
4. Do the checkpoint from memory. Then read Appendix C and argue with it.
5. Attempt the applied review questions in pairs. They are written to be discussed, not looked up.

⚠️ On the laboratories, and what they need

Every lab in this book runs on one student laptop with free software. Where the subject is an enterprise platform — vSphere, Hyper-V on a cluster, a Ceph estate, an NVIDIA vGPU — the chapter teaches the real thing and the lab uses a free equivalent that behaves the

same way, naming the substitution in its first line. KVM with `virsh` stands in for vSphere; `kind` stands in for a production Kubernetes cluster; LVM thin pools stand in for enterprise storage.

Three labs need hardware not every reader has: nested virtualization in Chapter 4, GPU partitioning in Chapter 20, and an SEV- or TDX-capable processor in Chapter 21. Each ships captured output in `code/` so the exercise can be done by reading. No lab in this book is an instruction to buy something.

Appendix A installs and verifies the whole stack in one sitting.

Course Specification

This page is the course file. It states what HEC requires of a course called *Virtual Systems and Services*, and shows where in this handout each requirement is met.

This course's position in the HEC curricula is unusual enough to state plainly before the tables begin. HEC published contents for it *once*, in the 2017 *Curriculum of BS & MS CS, SE & IT* (pp. 71–130). The 2023 revision keeps the title in the BSIT tables as a domain elective but publishes no contents for the Information Technology cluster at all. The 2025 revision removes the course, dividing its ground between *Cloud Computing*, a new major core, and *Microservices Architecture and Docker Containers*.

The 2017 outline is therefore the only authoritative description that exists, and this handout treats it as binding on Parts I and II and as a starting point for the rest.

Course particulars

Item	Value
Course title	Virtual Systems and Services
Programme	BS Information Technology. A domain elective under HEC 2017 and HEC 2023; absent from HEC 2025
Credit hours	3 (2–1): two credit hours of lectures and one of laboratory
Prerequisite	Programming Fundamentals (HEC 2017). In practice this course also assumes comfort at a Linux command line; the self-check two pages on says what that means concretely
Duration	Sixteen teaching weeks, with a midterm in Week 8 and a project due in Week 16 (Appendix G)
Course introduction	The HEC 2017 aim, verbatim: “This course will investigate the current state of virtualization in computing systems. Virtualization at both the hardware and software levels will be examined, with emphasis on the hypervisor configurations of systems such as Vmware, Zen and Hyper-V. The features and limitations of virtual environments will be considered, along with several case studies used to demonstrate the configuration and management of such systems. Para-virtualized software components will be analyzed and their pros and cons discussed. Processor and peripheral support for virtualization will also be examined, with a focus on emerging hardware features and the future of virtualization.”
Set texts	See the note below. The two titles HEC 2017 lists are not about this subject; Appendix E gives the references this handout actually uses

The set texts in the published outline are the wrong books

HEC 2017 lists two texts for this course: K. M. Stanney, *Handbook of Virtual Environments: Design, Implementation and Applications*, and G. Burdea, *Virtual Reality Technology*.

Both are virtual-reality references — head-mounted displays, haptics, presence and simulator sickness. Neither mentions a hypervisor. The outline's own description is unambiguously

about system virtualization, so the reading list appears to have been assembled from the course title rather than its contents. This handout does not use them. Appendix E lists the primary sources it does use, beginning with Popek and Goldberg’s 1974 paper, the Intel and AMD architecture manuals, and the Xen, KVM and Firecracker papers.

The outline also spells Xen as “Zen”. It is Xen, after the Xenoserver project at Cambridge.

Course learning outcomes

HEC 2017 publishes an outline but no Bloom-levelled outcomes, and the later editions publish neither. The six below are written against the 2017 outline and traced to the chapters that deliver them.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Explain what a virtualization layer is, and classify a given system by the layer it virtualizes and the technique it uses	C2 Understand	1, 2, 8
CLO-2	State the Popek–Goldberg conditions and explain why x86 failed them, and how binary translation and hardware assistance each answer that failure	C3 Apply	3, 4
CLO-3	Configure and manage virtual machines and containers on at least three platforms, including storage and networking	C3 Apply	5–7, 9–13, 16
CLO-4	Analyze the performance of a virtualized workload, identify the bottleneck from hypervisor metrics, and size a host for a given set of guests	C4 Analyze	6, 17, 18
CLO-5	Evaluate the security and the cost of a virtualization or cloud design, and justify a deployment model against stated requirements	C5 Evaluate	14, 19, 21, 22
CLO-6	Build, deploy, measure and document a working virtualized service	C6 Create	23, and every lab

Coverage of the HEC 2017 course outline

The published outline is a single descriptive paragraph rather than a topic list. Each of its eleven clauses is traced below, in the order HEC writes them. Nothing is omitted.

HEC 2017 clause	Chapter	Where exactly
“The current state of virtualization in computing systems”	1	What the layer is, the four motives for inserting one, and the line from CP-67 to the cloud
“Virtualization at the hardware level”	4	VT-x and AMD-V, the VMCS, extended page tables, the IOMMU

HEC 2017 clause	Chapter	Where exactly
“... and software levels”	2, 3	Full virtualization, paravirtualization and binary translation; the virtual machine monitor itself
“Hypervisor configurations of systems such as VMware, Xen and Hyper-V”	5	All three, plus KVM, each as an architecture diagram and a comparison table
“The features ... of virtual environments”	6, 7, 17	Overcommit, ballooning, page sharing, NUMA; virtio and passthrough; snapshots and live migration
“... and limitations”	6, 18, 19	What overcommit costs, how a virtualized benchmark lies, and the attack surface the layer creates
“Several case studies ... configuration and management of such systems”	16, 22	The management platforms in Chapter 16; four full case studies in Chapter 22
“Para-virtualized software components will be analyzed”	7	The split-driver model, virtqueues, and the PV driver sets of each platform
“... and their pros and cons discussed”	7	Emulated against paravirtualized against passthrough, measured, with the cost of each
“Processor ... support for virtualization”	4	Chapter 4 entire
“... and peripheral support”	7, 20	SR-IOV and VFIO in Chapter 7; GPU and accelerator partitioning in Chapter 20
“Emerging hardware features and the future of virtualization”	20, 21	Accelerator virtualization; confidential computing, attestation, CXL and composable infrastructure

Topics taught here that the 2017 outline does not name

The 2017 outline predates Docker’s general adoption, Kubernetes entirely, and every confidential-computing platform in Chapter 21. Eight chapters go beyond its letter. They are listed here so that the departure is explicit.

Chapters	Why they are here	What they rest on
8–11 Containers	The dominant form of virtualization in industry since 2015, and the direct subject of the course HEC 2025 created to replace this one. Taught as what they are — operating-system-level virtualization, one layer above the hypervisor	HEC 2025, <i>Microservices Architecture and Docker Containers</i>
12–13 Storage and network	The 2017 clause “virtualization at both the hardware and software levels” is not restricted to the processor, and no hypervisor is useful without virtual disks and virtual switches	HEC 2017, “hardware and software levels”

Chapters	Why they are here	What they rest on
14 Cloud models	Where virtualization is actually consumed. Without it the course ends at the machine room door	HEC 2025, <i>Cloud Computing</i>
21 Confidential computing	The direct continuation of “emerging hardware features and the future of virtualization”, written sixteen years later	HEC 2017, “emerging hardware features”



The overlap with *Cloud Computing* is real, intended, and bounded. The curriculum map that traces these editions reaches a blunt conclusion — “virtualisation is one chapter of cloud computing, not a peer of it” — and where a department offers both courses, that is the right call.

Where this one is offered, it earns its three credit hours by going a layer *below* a cloud course rather than alongside it. A cloud course explains what an instance is and how to rent one. This course explains what happens on the host when you do: which instruction trapped, which page table was walked twice, which driver was split in half, and why the instance beside yours can make yours slow. Appendix F draws the line topic by topic, and names what this course deliberately does not teach.

The Course at a Glance

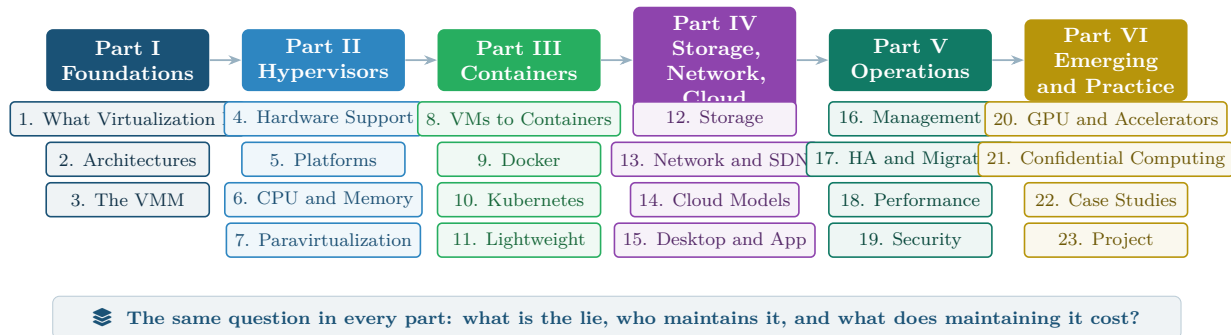


Figure F.1: The six parts and the chapters in each. Parts I and II teach the HEC 2017 outline close to specification; Parts III to VI carry it forward to the stack in production use today.

This course sits beside Cloud Computing, System and Network Administration and the CCNA track, and one layer below all three. Appendix F states what belongs to each.

The Laboratory Stack

This course is unusual among the BSIT electives in that the subject is infrastructure, and infrastructure is expensive. A faithful laboratory for the 2017 outline would need a vSphere licence, a Hyper-V cluster, a SAN and a GPU with vGPU entitlement — perhaps forty thousand dollars of software before any hardware.

That is not what this book asks for. Every lab runs on one laptop with free software, and the substitution is named each time it is made.

What the chapter teaches	What the lab uses	Why the substitution is honest
VMware ESXi and vSphere	KVM with <code>virsh</code> and <code>virt-manager</code>	Both are Type-1 hypervisors with the same primitives. The vocabulary differs; the mechanism does not
Microsoft Hyper-V	Hyper-V where Windows Pro is available, KVM otherwise	Hyper-V is free with the operating system most students already run
vMotion / Live Migration	<code>virsh migrate</code> between two nested hosts	Pre-copy migration with dirty-page tracking, which is the mechanism Chapter 17 teaches
Enterprise SAN, Ceph, vSAN	LVM thin pools; a single-node <code>cephadm</code> cluster	Thin provisioning, copy-on-write and the over-subscription failure are all reproducible on one disk
Production Kubernetes	<code>kind</code> (Kubernetes in Docker)	A real API server, scheduler and kubelet. Only the node count is a pretence
NVIDIA vGPU and MIG	<code>nvidia-smi</code> partitioning where a GPU exists; captured output otherwise	Partitioning policy is the lesson; the silicon is not available to rent by the hour
AWS, Azure and GCP	The three free tiers, with a teardown step in every lab	The instance you launch is the instance the chapter describes
Intel TDX, AMD SEV	Captured attestation output in <code>code/</code>	Confidential VMs need a specific processor generation and a cooperating host

⚠ One thing the free stack cannot teach you

Scale. Every pathology in Chapter 18 — the noisy neighbour, the oversubscribed cluster, the storage array that saturates at three in the afternoon — is a property of many machines sharing one resource, and a laptop running four guests will not show it to you honestly.

Where that is the case the chapter says so and gives measured numbers from real estates instead of asking you to produce them. Read those numbers as data, and treat the lab as the mechanism rather than the magnitude.

Appendix A installs the whole stack and verifies it. Appendix B is the command reference you will keep open beside the labs.

Notation and Conventions

Term or symbol	Meaning in this book
host	The physical machine, and the software running directly on it
guest	An operating system running inside a virtual machine
hypervisor, VMM	Used interchangeably, as the literature does. Where the distinction matters — KVM is a VMM inside a host kernel, not a free-standing hypervisor — the text says which it means
Ring 0, 1, 3	x86 privilege levels: kernel, the level Xen deprived guests into, and user
root, non-root	VMX operation modes, Chapter 4. Nothing to do with the <code>root</code> user
GVA, GPA, HPA	Guest virtual, guest physical and host physical address — the three address spaces a nested page walk crosses
vCPU	A virtual processor presented to a guest; scheduled onto a physical core by the hypervisor
pCPU	A physical core or hardware thread
R	Consolidation ratio: guests per host
σ	Overcommit ratio: allocated resource divided by physical resource
%RDY	Fraction of time a vCPU was runnable but not scheduled. The single most useful number in Chapter 18
RPO, RTO	Recovery point objective (how much data you may lose) and recovery time objective (how long you may be down)
p95, p99	The 95th and 99th percentile of a latency distribution
1 GiB, 1 GB	Binary and decimal gigabytes, 2^{30} and 10^9 bytes. Storage vendors mean the second; operating systems usually report the first. This book is explicit every time it matters



Two conventions worth stating because the industry is inconsistent about them.

Type-1 and Type-2 are used in this book as Goldberg defined them in 1973 — by whether the VMM runs on bare metal or on a host operating system — and Chapter 2 shows why the distinction is blurrier in practice than the textbook diagram suggests. KVM is the awkward case, and it is discussed rather than assigned.

Virtualization and emulation are not synonyms. Virtualization runs guest instructions on the real processor wherever it can; emulation interprets every one of them. QEMU does both, in different modes, which is why it appears on both sides of the comparison in Chapter 2.

Prerequisite Self-Check

The published prerequisite is *Programming Fundamentals*, which is not really what this course needs. What it needs is the twelve things below. If you can do ten of them, begin at Chapter 1. If you cannot, Appendix A and the first two sections of Appendix B will close most of the gap in an afternoon.

1. Explain what an operating system kernel does that a user program cannot. (Operating Systems)
2. Describe what happens on a system call, from the user instruction to the return. (Chapter 3)
3. Explain virtual memory: what a page table maps, and what the MMU does with it. (Chapter 4)
4. State what a process is, and what the kernel keeps for each one. (Chapter 8)
5. Navigate a Linux filesystem and read a file from the command line without a graphical tool. (Appendix B)
6. Edit a text file over SSH and explain what SSH is doing. (Appendix A)
7. Read a shell script containing a loop, a conditional and a pipeline. (Appendix B)
8. Explain the difference between an IP address, a subnet mask and a default gateway. (Chapter 13)
9. Say what a TCP port is and how two services share one host. (Chapter 13)
10. Convert between binary, decimal and hexadecimal, and read a size in bytes written either way. (Notation, above)
11. Explain what a file format is, given that a virtual disk is a file that contains a filesystem. (Chapter 12)
12. Describe what a compiler produces and in what sense a CPU “runs” it. (Programming Fundamentals)



Notice what is *not* on that list: no calculus, no statistics, no networking certification, and no prior exposure to any cloud platform. The arithmetic in this book is ratios, percentages and powers of two. What it genuinely demands instead is comfort with the idea that software can be *underneath* other software — which is uncomfortable the first time, and then never again.

Contents

How to Use This Handout	ii
Course Specification	v
The Course at a Glance	ix
The Laboratory Stack	x
Notation and Conventions	xi
Prerequisite Self-Check	xii
I Foundations of Virtualization	1
1 What Virtualization Is	3
1.1 A Room Full of Idle Servers	3
1.2 What It Is Not	4
1.3 Why Anyone Would Do This	5
1.4 Where It Came From	7
1.5 What Is Virtualized, and Where to Find It	8
2 Virtualization Architectures	12
3 The Virtual Machine Monitor	13
II Hypervisors and Compute Virtualization	14
4 Hardware Support for Virtualization	16
5 Major Hypervisor Platforms	17
6 CPU and Memory Virtualization	18
7 Paravirtualization and Device Virtualization	19
III Containers and Lightweight Virtualization	20
8 From Virtual Machines to Containers	22
9 Docker and the Container Ecosystem	23
10 Container Orchestration with Kubernetes	24

11 Lightweight and Secure Virtualization	25
IV Storage, Network and Cloud Virtualization	26
12 Storage Virtualization	28
13 Network Virtualization and SDN	29
14 Cloud Service and Deployment Models	30
15 Desktop and Application Virtualization	31
V Management, Performance and Security	32
16 Virtualization Management Platforms	34
17 High Availability, Live Migration and Disaster Recovery	35
18 Performance Monitoring and Capacity Planning	36
19 Security in Virtualized Environments	37
VI Emerging Hardware and Practice	38
20 GPU and Accelerator Virtualization	40
21 Confidential Computing and Emerging Hardware	41
22 Case Studies in Enterprise and Cloud Virtualization	42
23 The Semester Project	43
Appendices	44
A The Laboratory Toolkit	45
B Command Reference	46
C Answers to the Checkpoints	47
D Glossary	48
E Resources and Further Reading	49
F The Boundary with the Neighbouring Courses	50
G The Sixteen-Week Schedule	51

List of Figures

F.1	The six parts and the chapters in each. Parts I and II teach the HEC 2017 outline close to specification; Parts III to VI carry it forward to the stack in production use today.	ix
P1.1	Reading path through Part I. Chapter 3 is the hinge: it states the conditions a correct hypervisor must meet, and every hardware feature in Part II exists because x86 did not meet them.	2
1.1	The layer, and the only claim it makes. Each guest is told a different machine exists, and none of the three was modified to be told. Every later chapter replaces one row of this diagram with something more specific.	4
1.2	The same work, before and after. Nothing was made faster; the idle capacity was simply stopped from being bought. The right-hand picture is also the risk picture: three failure domains where there were forty.	7
P2.1	Reading path through Part II. Chapter 4 is the silicon; the three that follow spend it — on platforms, on the processor and memory, and on everything plugged into the bus.	15
P3.1	Reading path through Part III. The arc runs from kernel primitives to a cluster scheduler — and then back to Part II, because Chapter 11’s answer to container isolation is to put a virtual machine underneath each one.	21
P4.1	Reading path through Part IV. Chapters 12 and 13 complete the virtual machine; Chapter 14 is where the industry sells it, and Chapter 15 is the case where the thing virtualized is somebody’s desk.	27
P5.1	Reading path through Part V. Live migration is the one in the middle because it is what the rest is for: a fleet you can manage is a fleet whose machines can move without stopping.	33
P6.1	Reading path through Part VI. Chapters 20 and 21 are where the hardware is still moving; Chapters 22 and 23 put the whole book to work, first on other people’s decisions and then on your own.	39

PART I

Foundations of Virtualization

Every idea in this book is a special case of one move: put a layer underneath something, and make what sits above believe nothing has changed. Part I says what that layer is, names the ways it can be built — and then proves the uncomfortable fact that for twenty years the most widely sold processor in the world could not host one correctly.

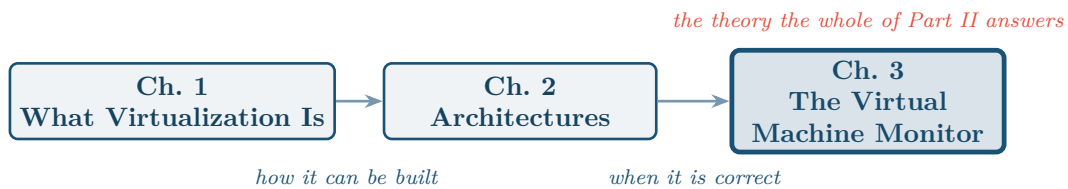


Figure P1.1: Reading path through Part I. Chapter 3 is the hinge: it states the conditions a correct hypervisor must meet, and every hardware feature in Part II exists because x86 did not meet them.

What Virtualization Is

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Virtualization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** virtualization as a layer that presents a resource different from the one it sits on, and **identify** that layer in a system you are shown.
- **Distinguish** a virtual machine from a process, an emulator and a simulator, by what each one is allowed to assume about the hardware.
- **Explain** the four reasons an organisation inserts such a layer — consolidation, isolation, encapsulation and elasticity — and **state** which of the four motivated each milestone in the field's history.
- **Compute** a consolidation ratio and the power it saves, from measured utilisation figures.
- **Describe** what is being virtualized in a given product, and locate the chapter of this book that covers it.

🔗 Before You Start

Nothing in this book. You need the idea that an operating system stands between a program and the hardware, and that a program cannot tell the difference between a real device and a convincing imitation of one. If the second half of that sentence sounds improbable, this chapter is the right place to start.

📖 Key Terms in This Chapter

virtualization • virtual machine • hypervisor • virtual machine monitor • host • guest • abstraction • multiplexing • aggregation • emulation • consolidation • consolidation ratio • utilisation • server sprawl • isolation • encapsulation • elasticity • time-sharing

1.1 A Room Full of Idle Servers

Walk into a university machine room that has not been consolidated and you will find something strange: forty servers, each costing as much as a car, each running at eight per cent of its capacity. The library catalogue has a server. The examination system has a server. The staff portal has a server. None of them is busy. Together they draw fourteen kilowatts and need as much again in cooling, and the department pays for all of it.

Nobody planned this. It arrived one purchase order at a time, and each purchase was defensible. The examination system must not be slowed down by the library catalogue, so it gets its own machine. The portal needs a newer version of the database than the catalogue can tolerate, so it gets its own machine. The vendor's support contract is void if anything else runs on the box, so it gets its own machine. Every reason is a reason about *isolation*, and the only isolation mechanism available was a second computer.

This is **server sprawl**, and it is the problem virtualization was brought to industry to solve. The insight is that the forty applications genuinely do need to be isolated from one another — that part was never wrong — but that isolation does not have to be purchased one chassis at a time.

Virtualization

Virtualization is the insertion of a software layer that presents a resource which differs from the one physically present, while preserving the interface that software above the layer already expects.

The layer does one of three things, or several at once:

- **multiplexing** — one physical resource is presented as many (one processor as twelve vCPUs);
- **aggregation** — many physical resources are presented as one (twelve disks as one volume);
- **abstraction** — a resource is presented as a different kind of thing entirely (a file on a disk presented as a disk).

What makes it virtualization rather than merely software is the second clause: the interface is *preserved*. An unmodified operating system, written with no knowledge that any of this is happening, runs correctly above the layer.

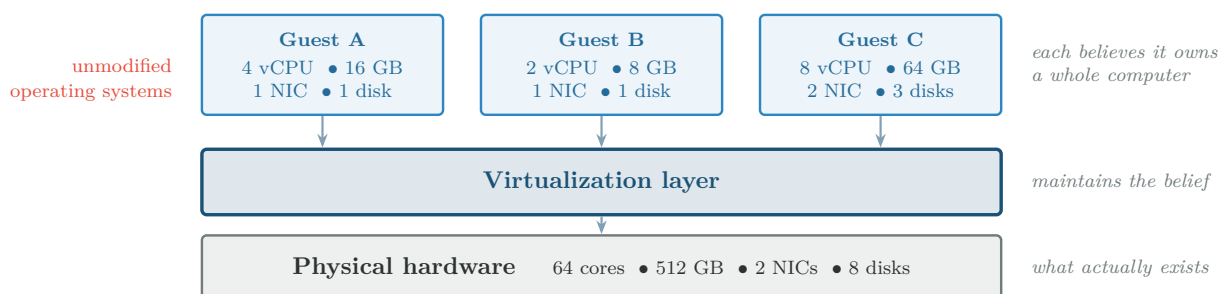


Figure 1.1: The layer, and the only claim it makes. Each guest is told a different machine exists, and none of the three was modified to be told. Every later chapter replaces one row of this diagram with something more specific.

Virtual Machine, Hypervisor, Host, Guest

A **virtual machine** is the illusion the layer sells: a complete computer — processor, memory, firmware, disks, network interfaces — implemented in software, on which an operating system can be installed as if on metal.

The **hypervisor**, equivalently the **virtual machine monitor** or VMM, is the software that creates and maintains that illusion. The **host** is the physical machine and the software running directly on it; a **guest** is an operating system running inside a virtual machine.

This book uses *hypervisor* and *VMM* interchangeably, as the literature does. Chapter 3 is the one place the distinction is sharpened, because that is where it begins to matter.

1.2 What It Is Not

Three neighbouring ideas are routinely confused with virtualization, and the confusion is worth clearing before it compounds.

Virtualization, Emulation, Simulation

Emulation interprets every guest instruction in software, translating from one instruction set to another. An ARM emulator on an x86 laptop reads an ARM instruction, works out what it means, and performs the equivalent x86 work. It is slow — commonly ten to a hundred times slower than native — and completely general: the guest architecture need not resemble the host at all.

Virtualization runs guest instructions *on the real processor* wherever it safely can, and intervenes only for the instructions it cannot allow to execute directly. It is therefore fast — commonly within a few per cent of native — and restricted: the guest must be built for the same architecture as the host.

Simulation models a system’s behaviour without claiming to run its software: a network simulator predicts throughput; it does not boot a router.

QEMU does the first two in different modes, which is why it appears on both sides of the comparison in Chapter 2. Running `qemu-system-aarch64` on an x86 host is emulation. Running `qemu-system-x86_64` with KVM enabled is virtualization, and the same binary does both.

⚠ A container is not a virtual machine, and both are virtualization

The two most common statements about containers in a first course contradict each other, and both are half right.

A container does not contain a virtual machine: there is no virtual processor, no virtual firmware, no guest kernel. Processes in a container run on the host’s kernel as ordinary processes, and `ps` on the host can see them.

But a container *is* virtualization by the definition above. The layer is inserted inside the kernel rather than beneath it, and what it multiplexes is the operating system’s own namespace of process identifiers, mount points, network interfaces and users. The interface preserved is the system-call interface, not the instruction set. Chapter 8 makes this precise; the reason it matters this early is that “virtual machine or container?” is a question about *which layer*, not about whether a layer exists.

1.3 Why Anyone Would Do This

Four motives, and they arrived in this order historically. Every product in this book was built for at least one of them, and the ones that lasted serve several.

Motive	What it buys	Where this book shows it
Consolidation	Many lightly-loaded workloads on one well-loaded machine. Fewer machines, less power, less cooling, less floor space, less to patch	The worked example below; Chapter 6 on how far it can be pushed; Chapter 18 on when it has gone too far
Isolation	A fault, a crash or a compromise in one workload does not reach the others. This is the reason the forty servers existed, and virtualization preserves it while removing the forty	Chapter 3 on what makes isolation sound; Chapter 19 on where it fails
Encapsulation	A running machine becomes a file. It can be copied, versioned, moved, snapshotted and restored — operations no physical server supports	Chapter 12 on the file; Chapter 17 on moving it while it runs
Elasticity	Capacity becomes a quantity you request rather than a chassis you procure. Minutes instead of months, and returnable	Chapter 14 entire; Chapter 10 for the same idea one layer up

💡 Encapsulation is the one with no physical analogue

Consolidation is an efficiency argument and isolation is a safety argument, and a sufficiently rich organisation could buy its way out of both. Encapsulation is different in kind.

Once a running computer is a file, every operation you know how to perform on a file becomes available to it. You can take a snapshot before a risky upgrade and return to it in four seconds when the upgrade fails. You can copy a production machine into a test environment, bit for bit, including the fault you are trying to reproduce. You can keep last Tuesday. None of this has a physical equivalent — there is no way to take a snapshot of a server in a rack — and it is the capability administrators miss most when they go back.

📊 Worked Example: The Forty Servers, Costed

The machine room of the opening section, measured over one semester. Decide how many hosts replace it, and what that saves.

What is there. 40 servers, each 2 sockets $\times$ 8 cores = 16 cores and 64 GB of memory. Measured per server: mean CPU utilisation 8%, peak 22%; mean memory in use 11 GB. Each draws 350 W.

Step 1 — what is actually consumed.

- CPU: $40 \times 16 \times 0.08 = 51.2$ cores' worth of work, spread over 640 physical cores.
- Memory: $40 \times 11 = 440$ GB in use, of 2,560 GB installed.

Step 2 — size the replacement on memory, not CPU. Memory is the binding constraint in almost every consolidation, because it cannot be oversubscribed as freely as processor time (Chapter 6 says why). Allow 30% headroom for growth and for the hypervisor itself:

$$440 \times 1.3 = 572 \text{ GB}$$

On hosts of 512 GB that is $\lceil 572/512 \rceil = 2$ hosts. Add one more so that a single host failure does not take the estate down — an $N+1$ design — giving **3 hosts**.

Step 3 — check that CPU is not the real constraint. Give each workload 4 vCPUs, which is generous against a measured peak of $16 \times 0.22 = 3.5$ cores. Three hosts of 2×32 cores give 192 physical cores:

$$\sigma = \frac{40 \times 4}{192} = \frac{160}{192} = 0.83$$

An overcommit ratio below 1 means that even if every guest demanded its full allocation simultaneously, the cores exist. CPU is not the constraint; memory was right.

Step 4 — the consolidation ratio.

$$R = \frac{40 \text{ workloads}}{3 \text{ hosts}} = \mathbf{13.3 : 1}$$

Step 5 — the power. The new hosts are larger and draw 700 W each.

$$40 \times 350 = 14,000 \text{ W} \quad 3 \times 700 = 2,100 \text{ W} \quad \text{saved: } 11.9 \text{ kW}$$

Cooling roughly tracks the load; at a power usage effectiveness of 1.6 the facility saves $11.9 \times 1.6 = 19.0$ kW. Over a year:

$$19.0 \times 8,760 = 166,400 \text{ kWh}$$

At PKR 45 per kWh that is **about PKR 7.5 million a year**, before counting the thirty-seven servers not bought, the rack space not rented, or the thirty-seven operating systems not patched.

What the number does not say. Those three hosts are now a shared fate. The arithmetic above bought a 13:1 reduction in machines and a 13:1 concentration of risk, and Chapter 17 is where that bill comes due.

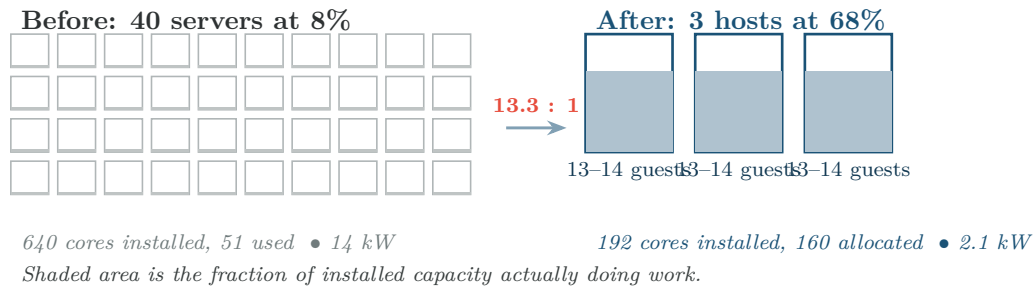


Figure 1.2: The same work, before and after. Nothing was made faster; the idle capacity was simply stopped from being bought. The right-hand picture is also the risk picture: three failure domains where there were forty.

1.4 Where It Came From

Virtualization is not a 2000s invention. It is a 1960s invention that the personal computer made irrelevant for thirty years and the data centre made urgent again.

IBM built CP-67 for the System/360 Model 67 because its customers had a problem of scarcity rather than sprawl. A mainframe cost millions and there was exactly one; a dozen departments each needed their own operating environment on it, and several of those operating systems could not be persuaded to share. CP-67 gave each department a complete virtual System/360 of its own. Its successor VM/370 ran in production for decades, and its descendant z/VM still does.

Then the minicomputer and the personal computer made computers cheap, and the problem CP-67 solved stopped being a problem: if you needed a second computing environment you bought a second computer. The idea went quiet for twenty-five years, kept alive on mainframes and in research, while the architecture that won the market — x86 — turned out, for reasons Chapter 3 sets out exactly, to be incapable of supporting it correctly.

💡 Why the second wave looked nothing like the first

CP-67 was built for scarcity: one machine, many users. VMware was built for the opposite — forty machines, each almost idle. The motive inverted completely, and the mechanism was reused unchanged.

That is worth noticing because the same inversion happened again. Containers were adopted in 2013 largely for *packaging*: a reproducible image that runs the same on a laptop and in production. Only afterwards did the industry discover that the same mechanism gave it density — a thousand containers on a host that would hold fifty virtual machines — and density became the reason people cite today. A mechanism outlives the motive that produced it, which is why this book teaches mechanisms.

1.5 What Is Virtualized, and Where to Find It

“Virtualization” names a technique, not a product category. The useful question about any system is *which resource* it virtualizes, because that determines what it can promise.

What is virtualized	So that	Examples	Chapter
The whole machine	An unmodified operating system boots on it	ESXi, Hyper-V, KVM, Xen	Chapter 5
The operating system	Processes see a private system, sharing one kernel	Docker, containerd, LXC	Chapter 8, Chapter 9
The processor and memory	Several guests divide cores and RAM	vCPU scheduling, ballooning	Chapter 6
A device	A guest drives hardware it does not exclusively own	virtio, SR-IOV, VFIO	Chapter 7
Storage	A file behaves as a disk; many disks behave as one	qcow2, VMDK, Ceph, vSAN	Chapter 12
The network	A switch exists in software; a LAN spans data centres	Open vSwitch, VXLAN, SDN	Chapter 13
The desktop	The screen is here and the computer is elsewhere	VDI, Citrix, AVD	Chapter 15
An accelerator	One GPU serves several tenants	vGPU, MIG, SR-IOV	Chapter 20

! Common Pitfalls

- **Treating “virtual” as meaning “not real”.** A virtual processor executes real instructions on real silicon and consumes real electricity. What is virtual is the *mapping*, not the work. The fastest way to mislead yourself about capacity is to believe the vCPU count is free.
- **Sizing a consolidation from installed capacity rather than measured use.** The forty servers above had 2,560 GB installed and used 440 GB. Planning against the first number buys five times the hardware you need; it is the single most common and most expensive mistake in this subject.
- **Assuming consolidation is free.** It converts capital cost into shared risk and contention. Three hosts instead of forty is also three failure domains instead of forty, and every guest on a host is now a potential noisy neighbour to every other (Chapter 18).
- **Calling a hypervisor an operating system, or vice versa.** They schedule different things for different consumers. The confusion makes Chapter 3 incomprehensible, because the whole chapter turns on what the hypervisor is allowed to assume that an operating system is not.
- **Believing the marketing distinction between “virtualization” and “cloud”.** A cloud is virtualization with a billing system, an API and somebody else’s staff. Chapter 14 is explicit about which of the three is doing the work in any given claim.

🔧 Try It Yourself: Your First Virtual Machine

Runs on any Linux host, or on Windows under WSL2 with VirtualBox substituted at step 2. Appendix A installs the prerequisites. Expect fifteen minutes.

```
#!/usr/bin/env bash
# ch01_first_vm.sh -- create, snapshot, break and restore a virtual machine.
set -euo pipefail

# --- 1. Does this processor support virtualization at all? -----
# vmx = Intel VT-x, svm = AMD-V. Chapter 4 explains what these flags buy.
# A count of 0 means the feature is absent or disabled in firmware.
grep -c -E '\<(vmx|svm)\>' /proc/cpuinfo
lscpu | grep -i 'virtuali[sz]ation' || echo "no virtualization line reported"

# --- 2. Install the hypervisor and confirm it is usable -----
sudo apt-get install -y qemu-kvm libvirt-daemon-system virtinst \
    libguestfs-tools cloud-image-utils
# virt-host-validate prints PASS/WARN/FAIL for every prerequisite.
sudo virt-host-validate qemu | head -20

# --- 3. Fetch a cloud image and give it a login -----
# A cloud image is a disk file with an operating system already installed:
# encapsulation (section 1.3) before you have even started.
wget -nc -O ubuntu.img \
    https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img
cat > user-data <<'YAML'
#cloud-config
password: vss2026
chpasswd: { expire: False }
ssh_pwauth: True
YAML
cloud-localds seed.iso user-data

# --- 4. Create the virtual machine -----
# Read the arguments as a parts list for an imaginary computer.
sudo cp ubuntu.img /var/lib/libvirt/images/vss1.qcow2
sudo qemu-img resize /var/lib/libvirt/images/vss1.qcow2 10G
sudo virt-install --name vss1 --memory 2048 --vcpus 2 \
    --disk /var/lib/libvirt/images/vss1.qcow2,format=qcow2 \
    --disk seed.iso,device=cdrom --os-variant ubuntu22.04 \
    --import --graphics none --noautoconsole

# --- 5. Look at it from the host -----
# dominfo is the guest as the hypervisor sees it: an allocation, not a machine.
sudo virsh list --all
sudo virsh dominfo vss1

# --- 6. Snapshot it, then break it on purpose -----
sudo virsh snapshot-create-as vss1 clean "before I broke it"
sudo virsh console vss1      # log in as ubuntu / vss2026, then:
                             # sudo rm -rf /etc/systemd ; sudo reboot
                             # Ctrl-] returns you to the host.

# --- 7. Undo four minutes of damage in four seconds -----
# No physical server supports this operation. That is section 1.3's point.
sudo virsh snapshot-revert vss1 clean --running
sudo virsh snapshot-list vss1

# --- 8. Tear down -----
sudo virsh destroy vss1 || true
sudo virsh undefine vss1 --remove-all-storage --snapshots-metadata
```

What to notice. Step 6 took longer to type than step 7 took to undo, and step 7 has no physical equivalent. Everything else in this book is detail; that asymmetry is the subject.

? Checkpoint — Test Yourself

1. A department runs 24 servers, each with 128 GB of memory, of which a measured 19 GB is in use. You are sizing hosts with 384 GB each, with 30% headroom and an N+1 spare. How many hosts, and what consolidation ratio does that give?
2. A colleague says “containers are faster than virtual machines because there is no virtualization layer.” Which half of that sentence is wrong, and what is the correct statement?
3. Of the four motives in Section 1.3, which one has no equivalent for physical servers at all? Give an operation it makes possible and say why a physical machine cannot offer it.

✓ Chapter Summary

- Virtualization inserts a layer that presents a resource differing from the physical one, while preserving the interface above it. It multiplexes, aggregates or abstracts — often all three.
- The preserved interface is what distinguishes it: an unmodified operating system runs above the layer.
- Emulation interprets every instruction and is general and slow; virtualization executes most instructions natively and is restricted and fast. QEMU does both, in different modes.
- Four motives: consolidation, isolation, encapsulation, elasticity. Encapsulation — a running machine as a file — is the one with no physical analogue.
- Size a consolidation from measured use, not installed capacity, and expect memory rather than CPU to be the binding constraint.
- The technique is from 1967. The twenty-five-year gap in the middle of its history is an x86 problem, and Chapter 3 is what went wrong.
- Ask of any product: *which resource does it virtualize?* That determines what it can promise, and which chapter of this book covers it.

✍ Review Questions

1. **(MCQ)** The defining property that separates virtualization from ordinary resource-sharing software is that: (a) it runs in kernel mode; (b) it preserves the interface, so unmodified software runs above it; (c) it improves performance; (d) it requires hardware support.
2. **(MCQ)** A tool runs ARM binaries on an x86 laptop at roughly one fiftieth of native speed. This is: (a) virtualization; (b) paravirtualization; (c) emulation; (d) containerisation.
3. **(Short)** Give the three things a virtualization layer can do to a resource, with one concrete example of each from a system you have used.
4. **(Short)** Explain why memory, rather than processor time, is usually the constraint that decides how many guests fit on a host.
5. **(Short)** The opening section argued that forty separate servers was a rational response to a real requirement. Name the requirement, and explain precisely what virtualization changed about it.
6. **(Applied)** A faculty runs 60 servers at a measured mean of 6% CPU and 14 GB of 96 GB memory each, drawing 300 W apiece. Size a replacement estate on 768 GB hosts drawing 850 W, with 30% headroom and N+1. Give the host count, the consolidation ratio and the annual energy saved at PUE 1.6 and PKR 45 per kWh. Then state the one risk your proposal has introduced and what you would do about it.
7. **(Applied)** For each of these, say which resource is virtualized and which chapter of this book covers it: a laptop running Windows inside a window on macOS; a Kubernetes cluster; an

NVMe disk presented to eight tenants; an office where staff keep no local files and log in to the same desktop from any machine; a GPU shared between three training jobs.

Virtualization Architectures

Part 1	Part 2	Part 3	Part 4	Part 5	Part 6
<i>Part I — Foundations of Virtualization</i>					

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

The Virtual Machine Monitor

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations of Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART II

Hypervisors and Compute Virtualization

Part I ended with a problem: the architecture could not be virtualized, and the software workarounds were slow and fragile. Part II is the answer, in the order the industry found it — first in silicon, then in four platforms built on that silicon, then in the two resources every guest competes for, and finally in the devices the guest thinks it owns.

the answer to Chapter 3

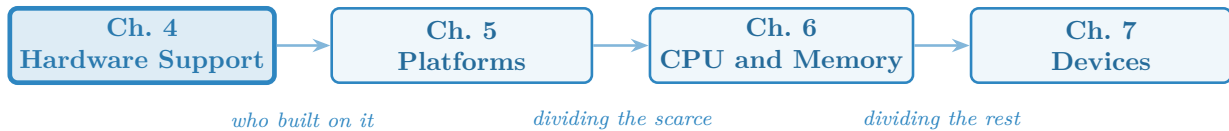


Figure P2.1: Reading path through Part II. Chapter 4 is the silicon; the three that follow spend it — on platforms, on the processor and memory, and on everything plugged into the bus.

Hardware Support for Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Major Hypervisor Platforms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

CPU and Memory Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Paravirtualization and Device Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Hypervisors and Compute Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART III

Containers and Lightweight Virtualization

A virtual machine virtualizes the hardware, and pays for a whole operating system to do it. Part III asks a cheaper question: if the processes already share one kernel, what would it take to stop them seeing each other? The answer turns out to be two features Linux already had — and an industry grew out of combining them.

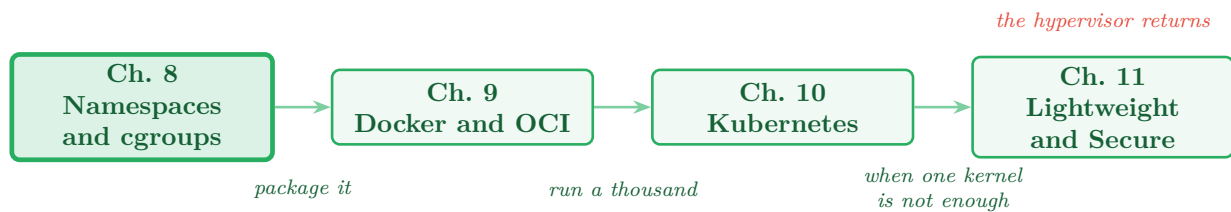


Figure P3.1: Reading path through Part III. The arc runs from kernel primitives to a cluster scheduler — and then back to Part II, because Chapter 11’s answer to container isolation is to put a virtual machine underneath each one.

From Virtual Machines to Containers

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Docker and the Container Ecosystem

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Container Orchestration with Kubernetes

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Lightweight and Secure Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Containers and Lightweight Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART IV

Storage, Network and Cloud Virtualization

A virtual machine with no disk and no network is a thought experiment. Part IV virtualizes everything that is not a processor — storage, then the network, then the whole arrangement, sold by the hour. It ends at the cloud, which is not a new idea in this book but the previous thirteen chapters with a billing system attached.

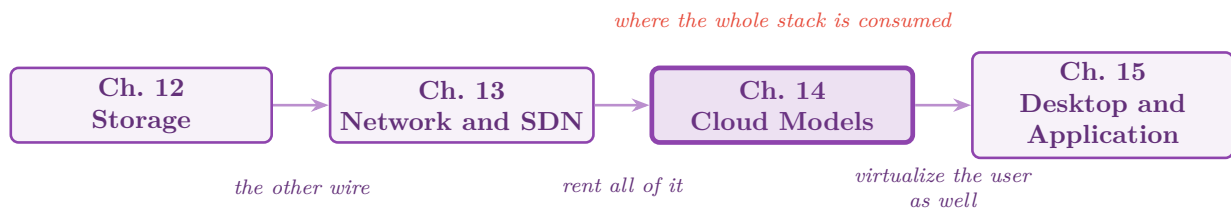


Figure P4.1: Reading path through Part IV. Chapters 12 and 13 complete the virtual machine; Chapter 14 is where the industry sells it, and Chapter 15 is the case where the thing virtualized is somebody's desk.

Storage Virtualization

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Network Virtualization and SDN

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Cloud Service and Deployment Models

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Desktop and Application Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Storage, Network and Cloud Virtualization

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART V

Management, Performance and Security

Everything so far has been how to build the layer. Part V is how to live with one. Fleets are managed, not configured; machines move between hosts while serving traffic; hosts run out of something before they run out of everything, and knowing which is a skill. And the layer that gave you all of this is also a piece of software with an attack surface.

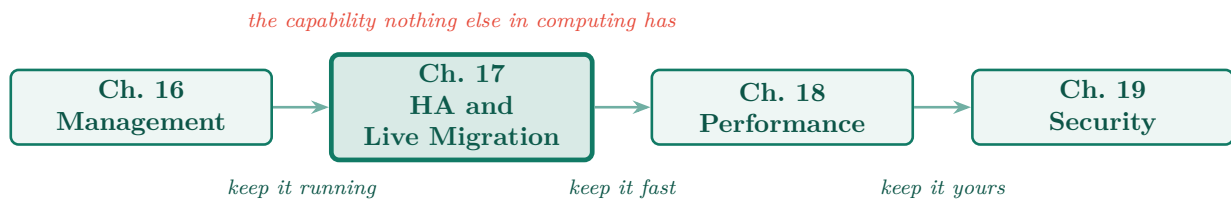


Figure P5.1: Reading path through Part V. Live migration is the one in the middle because it is what the rest is for: a fleet you can manage is a fleet whose machines can move without stopping.

Virtualization Management Platforms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Management, Performance and Security

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

High Availability, Live Migration and Disaster Recovery

Part 1	Part 2	Part 3	Part 4	Part 5	Part 6
--------	--------	--------	--------	---------------	--------

Part V — Management, Performance and Security

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Performance Monitoring and Capacity Planning

Part 1	Part 2	Part 3	Part 4	Part 5	Part 6
--------	--------	--------	--------	---------------	--------

Part V — Management, Performance and Security

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Security in Virtualized Environments

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART VI

Emerging Hardware and Practice

The 2017 outline ends with “emerging hardware features and the future of virtualization”. Two things have emerged since. Accelerators became the scarce resource and had to be divided like processors before them; and the trust model inverted — the guest stopped trusting the host, and the hardware had to be taught to take its side.

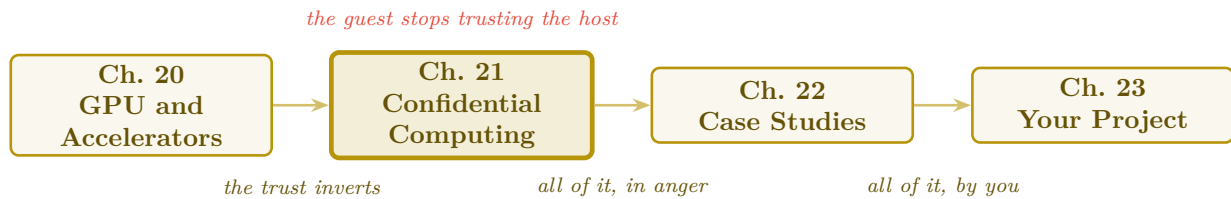


Figure P6.1: Reading path through Part VI. Chapters 20 and 21 are where the hardware is still moving; Chapters 22 and 23 put the whole book to work, first on other people’s decisions and then on your own.

GPU and Accelerator Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Confidential Computing and Emerging Hardware

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Case Studies in Enterprise and Cloud Virtualization

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

The Semester Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Emerging Hardware and Practice

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

Appendices

The Laboratory Toolkit

 Not written yet

This appendix is a placeholder.

Command Reference

 Not written yet

This appendix is a placeholder.

Answers to the Checkpoints

 Not written yet

This appendix is a placeholder.

Glossary

 Not written yet

This appendix is a placeholder.

Resources and Further Reading

 Not written yet

This appendix is a placeholder.

The Boundary with the Neighbouring Courses

 Not written yet

This appendix is a placeholder.

The Sixteen-Week Schedule

 Not written yet

This appendix is a placeholder.